

Manuel NSI Première - livret remédiation

Édition de travail 2026-2027

Table des matières

1	Représentation des données : types et valeurs de base	1
2	Types construits : p-uplets, tableaux, dictionnaires	2
3	Traitement de données en tables	4
4	Langages et programmation	5
5	Algorithmique 1 : parcours et tris	6
6	Algorithmique 2 : dichotomie, algorithmes gloutons, k plus proches voisins	7
7	Interactions entre l'homme et la machine sur le Web	8
8	Architecture de von Neumann et systèmes d'exploitation	9
9	Réseaux : transmission de données	10
10	Projet et méthodes	11

1 Représentation des données : types et valeurs de base

Exercice 1

🕒 10 min

Un élève écrit une boucle qui ajoute 0,1 trois fois de suite à une variable `x` initialisée à 0,0, puis teste `if x == 0.3`. Il s'attend à ce que ce test soit vérifié, mais ce n'est pas le cas.

1. Quelle est la valeur exacte affichée par `print(x)` après la boucle ?
2. Pourquoi le test `x == 0.3` échoue-t-il malgré tout ?
3. Proposer une correction du test qui fonctionne.

2 Types construits : p-uplets, tableaux, dictionnaires

Remédiation — Types construits

R1 — Je confonds tuple et liste (C1)

Rappel : un tuple est entouré de parenthèses () et est non mutable ; une liste est entourée de crochets [] et est mutable.

⌚ 5 min

Exercice 1

Pour chaque ligne, indique si la variable est un tuple ou une liste, et si on peut modifier son contenu :

```
1 a = (1, 2, 3)
2 b = [1, 2, 3]
3 c = ("hello",)
4 d = []
```

R2 — Je ne sais pas parcourir un tableau (C2)

Rappel : pour parcourir les éléments, écris `for element in tableau`. Pour parcourir les indices, écris `for i in range(len(tableau))`.

⌚ 8 min

Exercice 2

Écris une fonction `somme(tableau)` qui calcule la somme des éléments d'un tableau de nombres, sans utiliser la fonction `sum`.

```
»> somme([3, 7, 2])
12
```

R3 — Je confonds lignes et colonnes dans une grille (C3)

Rappel : dans `grille[i][j]`, `i` est le numéro de ligne et `j` le numéro de colonne.

⌚ 5 min

Exercice 3

On donne `g = [[10, 20, 30], [40, 50, 60]]`. Donne la valeur de :

1. `g[0][2]`
2. `g[1][0]`
3. `len(g)` et `len(g[0])`

R4 — J'obtiens `KeyError` avec un dictionnaire (C4)

Rappel : avant d'accéder à une clé, vérifie sa présence avec `if cle in dico` ou utilise `dico.get(cle, valeur_par_defaut)`.

Exercice 4

5 min

Corrige le programme suivant pour qu'il ne provoque pas d'erreur si la clé "age" est absente :

```
1 personne = {"nom": "Ali"}
2 print(personne["age"])
```

R5 — Je ne comprends pas pourquoi ma liste originale est modifiée (C5)

Rappel : `b = a` crée un alias (même objet en mémoire). Pour une copie indépendante, utilise `b = list(a)`.

Exercice 5

8 min

Le programme ci-dessous doit trier une copie sans modifier l'original. Corrige-le :

```
1 original = [5, 2, 8, 1]
2 copie = original
3 copie.sort()
4 print("Original :", original)
5 print("Trie :", copie)
```

Résultat attendu : l'original reste [5, 2, 8, 1].

3 Traitement de données en tables

⌚ 10 min

Exercice 1 ♦

Un élève cherche les élèves inscrits deux fois (même nom) dans la table suivante, mais compare des **lignes entières** au lieu de la seule colonne nom :

```
1 eleves = [  
2     {"nom": "Ali", "age": 16, "classe": "1A"},  
3     {"nom": "Ali", "age": 17, "classe": "1B"},  
4 ]  
5 lignes_str = [str(e) for e in eleves]  
6 doublons_naifs = [chaine for chaine in set(lignes_str) if lignes_str.count(chaine) > 1]
```

1. Que vaut doublons_naifs ? Le nom "Ali" est-il pourtant présent deux fois dans la table ?
2. Pourquoi la méthode de l'élève échoue-t-elle à détecter ce cas ?
3. Réécrire une détection de doublons qui compare uniquement la colonne nom, et vérifier qu'elle détecte bien "Ali".

4 Langages et programmation

Exercice 1

⌚ 10 min

Un élève écrit une fonction censée renvoyer le **minimum** d'une liste de nombres :

```
1 def minimum_bugue(liste):  
2     mini = 0  
3     for x in liste:  
4         if x < mini:  
5             mini = x  
6     return mini
```

1. Que renvoie `minimum_bugue([5, 3, 8])` ? Est-ce le résultat attendu ?
2. Pourquoi cette fonction se trompe-t-elle sur cet exemple, alors que tous les éléments de la liste sont positifs ?
3. Corriger la fonction en initialisant l'accumulateur correctement.

5 Algorithmique 1 : parcours et tris

⌚ 10 min

Exercice 1 ♦

Un élève écrit le tri par sélection suivant, mais place l'initialisation de `indice_min` à l'**intérieur** de la boucle `for` interne au lieu de l'externe :

```
1 def tri_selection_bugue(tableau):
2     tableau = tableau[:]
3     n = len(tableau)
4     for i in range(n - 1):
5         for j in range(i + 1, n):
6             indice_min = i # reinitialise a CHAQUE tour de la boucle interne
7             if tableau[j] < tableau[indice_min]:
8                 indice_min = j
9             tableau[i], tableau[indice_min] = tableau[indice_min], tableau[i]
10    return tableau
```

1. Tester cette fonction sur `[9, 1, 5, 3, 7]`. Le résultat est-il correctement trié ?
2. Expliquer précisément pourquoi remplacer `indice_min = i` à l'intérieur de la boucle `for j` empêche l'algorithme de retenir le minimum **global** de la partie restante.
3. Corriger le code en plaçant l'initialisation au bon endroit.

6 Algorithmique 2 : dichotomie, algorithmes gloutons, k plus proches voisins

Exercice 1

⌚ 10 min

Un élève écrit la recherche dichotomique suivante, avec une erreur dans la mise à jour de borne_inf :

```
1 def recherche_dichotomique_bugue(tableau_trie, valeur):
2     borne_inf, borne_sup = 0, len(tableau_trie) - 1
3     while borne_inf ≤ borne_sup:
4         milieu = (borne_inf + borne_sup) // 2
5         if tableau_trie[milieu] == valeur:
6             return milieu
7         elif tableau_trie[milieu] < valeur:
8             borne_inf = milieu # au lieu de milieu + 1
9         else:
10            borne_sup = milieu - 1
11    return -1
```

1. Sur le tableau $[1, 3, 5, 7, 9, 11, 13]$, que se passe-t-il si on cherche la valeur 4 (absente) avec cette fonction bugée ? (on pourra limiter le nombre d'itérations testées à 20 pour éviter une exécution trop longue)
2. Calculer le variant $V = \text{borne_sup} - \text{borne_inf}$ à chaque itération. Décroît-il toujours strictement avec cette version bugée ?
3. Corriger le code pour que la terminaison soit à nouveau garantie.

7 Interactions entre l'homme et la machine sur le Web

⌚ 10 min

Exercice 1 ♦

Un élève conçoit un formulaire de réinitialisation de mot de passe avec la méthode GET :

```
<form action="/nouveau_mdp" method="GET">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

1. En utilisant urlencode, construire l'URL qui serait générée si l'utilisateur saisit "MonSecret2026" comme nouveau mot de passe.
2. Le nouveau mot de passe apparaît-il dans cette URL ? Où cette URL risque-t-elle d'être enregistrée (historique, journaux serveur) ?
3. Réécrire l'attribut method du formulaire pour corriger ce problème.

8 Architecture de von Neumann et systèmes d'exploitation

Exercice 1



⌚ 10 min

Un élève lit la permission "rwxrwxr—" et affirme que « le bloc du milieu (rwx) concerne les *autres* utilisateurs », alors qu'il concerne en réalité le *groupe*.

1. Rappeler l'ordre des trois catégories dans une chaîne de permissions.
2. En utilisant droit_accorde du cours, déterminer si le **groupe** peut écrire dans ce fichier, puis si les **autres** utilisateurs peuvent y écrire.
3. Ces deux réponses sont-elles identiques ? Qu'est-ce que cela montre sur l'erreur de l'élève ?

9 Réseaux : transmission de données

⌚ 10 min

Exercice 1 ♦

Le message "RESEauxNSI" est découpé en 4 paquets de taille maximale 3. Ils arrivent au destinataire dans l'ordre : paquet 3, puis 1, puis 0, puis 2. Un élève réassemble le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans les trier :

```
1 ordre_arrivee = [paquets[3], paquets[1], paquets[0], paquets[2]]
2 message_reconstitue = "".join(p["donnees"] for p in ordre_arrivee)
```

1. Que contient message_reconstitue avec cette méthode ? Est-ce le message d'origine ?
2. Pourquoi cette méthode échoue-t-elle ici, alors qu'elle fonctionnerait si les paquets arrivaient dans l'ordre 0, 1, 2, 3 ?
3. Corriger le réassemblage pour qu'il fonctionne quel que soit l'ordre d'arrivée.

10 Projet et méthodes

Exercice 1

🕒 10 min

Le programme suivant plante à l'exécution :

```
1 def message_bienvenue_bugue(nom, age):  
2     return "Bonjour " + nom + ", tu as " + age + " ans."  
3  
4 message_bienvenue_bugue("Lina", 16)
```

1. Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. En appliquant la démarche méthodique du cours, identifier précisément quelle sous-expression du return pose problème, et pourquoi.
3. Corriger la fonction (au moins deux méthodes sont possibles : conversion explicite, ou f-string).